

ASTR4004 Problem Set 2

[LJ Spriggs (u7916719)]

Note: Find github repo at this link.

1 git IN PRACTICE

[2] Being able to share code and collaborate on projects is essential in both astronomical research and data science. Use a GitHub repository to track your work throughout this assignment. Demonstrate that you can:

- initialise a repository and push content to the main branch;
- create and work on a branch with a descriptive name;
- make regular commits with meaningful commit messages;
- merge your branch back into main.

Maintain a clear directory structure with data in `data/` and figures in `figures/`. Submit the link to your GitHub repository and specify your main working branch if it differs from `main`.

I create the `autogit.sh` script that automates the creation/committing/branching/merging of my folder `/home/spriggs/ASTR4004/problems et2` on Motley to my github repo. This script is run every time I remember to run it, or when I finish a question.

Below are my safety and confirmation commands. This script expects three inputs: `<target-repo-name>` (the name of the repository on my **github page**); `<branch-name>` (the name of the branch I want to commit to); `<merge-condition>` (must be T or F; tells script whether to merge `<branch-name>` with `main` after commit). The script will exit with error status 1 if: not three inputs provided; the confirmation is not complete (user replies with anything other than y or Y).

```
bash
```

```
#!/bin/bash

# (set -e) close/exit script if smth fails (prevent corrupted commits)
# (set -u) force missing/undefined variables to return an error (prevent empty string substitution)
# (pipefail) force pipe functions to return an error if conditional cmd fails
# see https://medium.com/@nitin.satyan/fail-fast-fail-loud-set-euo-pipefail-isnt-optional-5
#      b8002593037
# for full description/justification
set -euo pipefail

# confirm correct number of inputs to avoid later errors
if [[ $# -ne 3 ]]; then
    echo "$0 requires <target-repo-name> <branch-name> <merge-condition [T/F]>."
    echo "Scripted aborted."
    exit 1
fi

## CONFIRM CMD LINE INPUTS ##

# save cmd line inputs to variables
reponame="$1"
branchname="$2"
merge_cond="$3" # T for merge; F for no merge

# github location of repo
gitdir="git@github.com:ljiainspriggs/${reponame}.git"

# local location of repo (current dir, ensure in problem set folder)
localdir="$PWD"

# print variables
echo "Local Repo: $localdir"
echo "Target Repo: $gitdir"
echo "Branch: $branchname"
echo "Merge to main? $merge_cond"

# confirm variable correct with user (input from user)
read -p "Are the above correct? [Y/N] " confirm

if [[ "$confirm" == "Y" || "$confirm" == "y" ]]; then
    echo "Proceeding..." # if Y or y, continue
else
    echo "Script aborted." # else, abort
```

```

    exit 1 # close/exit script in error state
fi

```

I then force the script to correctly call the github repo. The script will close with error status 1 if the repository cannot be found/accessed on github.

```

bash

# try to access github repo, discard cmd std output/err
# true if cmd successful (status 0), false else
if git ls-remote "$gitdir" &>/dev/null; then
    echo "$reponame exists."
else
    echo "$reponame not found. Script aborted."
    exit 1
fi

```

I check if my local repository exists yet (for full automation, so this can be used for later projects). If the local repository is already setup for my working directory, the script allows me to update any branch on my github repo. Else, I am forced to complete an initial commit to main. Permissions for this were setup using an SSH key between motley and my github account.

```

bash

# set variable to know whether this is the initial commit
initial_commit=true

# true if directory .git exists
if [[ -d ".git" ]]; then
    echo "Working dir in git repo."
    initial_commit=false
else # if local repo not found, clone github repo to folder of repo name
    echo "Working dir not in git repo. Creating git repo..."
    git init # create git dir
    git branch -M main # define main as main branch
    git remote add origin "$gitdir" # define github loc for origin
    touch .gitignore # create git ignore file
    touch README.md # create read me file
    echo "Working log for ASTR4004 assignments." > README.md # add beginning/placeholder text
    echo "Local git repo created."
    git add -A # stage/cache all new files
    git commit -m "Initial commit." # commit initially to main
    git push -u origin main # push to github
    echo "Initial commit completed."
fi

# if NOT initial commit, check for desired branch on repo
if [[ "$initial_commit" == false ]]; then

    ## CHECK BRANCH EXISTS ON GITHUB REPO ##

    # try to access github repo branch, discard cmd std output/err
    # true if cmd successful (status 0), false else
    if git ls-remote --exit-code --branches origin "$branchname" &>/dev/null; then
        git switch "$branchname" # switch to branch
        echo "$branchname exists."
    else
        echo "$branchname not found. Creating..."
        git switch main
        git switch -c "$branchname" # create and switch to branch
        git push -u origin "$branchname" # push branch to github repo
        echo "$branchname created."
    fi
fi

```

I commit all changes to all files to my local repo, and then to github.

```

bash

git add -A # stage/cache all file changes in local repo for commit

# identify differences between cached repo (diff --cached), mute outputs of changes (--quiet)
# true if no changes (status 0), false else
if git diff --cached --quiet; then
    echo "No changes detected."
else

```

```

echo "Changes detected."

read -p "Commit message: " commit_message # await input of commit message from user

git commit -m "$commit_message" # commit to local repo with commit message
git push # push local repo to github
echo "Successfully committed to github repo."
fi

```

If requested in command line inputs, the script can merge the current branch with main.

```

bash

# true if merge_cond set to "T"
if [[ "$merge_cond" == "T" || "$merge_cond" == "t" ]]; then
    echo "Proceeding with merge to main..."
    git switch main # switch to main branch
    git merge "$branchname" # merge branch name with current branch (main)
    git push origin main # push updated main branch to github repo
    echo "Merging completed."
else
    echo "Merge not requested."
fi

```

Script completion is confirmed with summary for easy readability.

```

bash

# confirm completion of script
echo "Automated commit process completed with the following."
echo "Local Repo: $localdir"
echo "Target Repo: $gitdir"
echo "Branch: $branchname"
echo "Merge to main? $merge_cond"

```

This script was made executable and was executed using the below commands. The 744 code makes the script `r/w/x` for the owner of the script and `r` for everyone else.

```

bash

chmod 744 autogit.sh
./autogit.sh ASTR4004-Problem-Set-2 main F

```

Required folders are created using the below commands.

```

bash

mkdir figures
touch figures/.gitkeep
mkdir data
touch data/.gitkeep

```

The github repo can be accessed at github.com/ljiainspriggs/ASTR4004-Problem-Set-2.

2 BRIGHT STARS AROUND M67 WITH ADQL

[6] A colleague is interested in the open cluster Messier 67 ($RA = 132.825$ deg, $Dec = 11.8$ deg) and is considering an observation proposal using the 2dF fibre positioner and HERMES spectrography (effective for Gaia $G < 14$) to follow up stars observed with the APOGEE spectrograph, for which they only have 2MASS catalogue identifiers.

Your task is to assist by querying Gaia DR3 and performing some basic analysis.

- (a) Write an ADQL query that returns all stars in the Gaia DR3 `gaiadr3.gai_source` table within 1 deg of M67 with $G < 14$ (`phot_g_mean_mag` and crossmatches them with 2MASS (`gaiadr1.tmass_original_valid`). You can execute the query on the Gaia archive website or via ASTROQUERY. Include your ADQL query in the notebook and report the number of returned stars.

Begin by reading in necessary packages.

Python

```

# packages handling arrays
import pandas as pd
# package for data visualisation
import matplotlib.pyplot as plt
# astroquery for accessing the Gaia archive
from astroquery.gaia import Gaia

```

My xmatch from `gaiadr3.gaiia_source` to `gaiadr1.tmass_original_valid` goes via provided tables on the Gaia archive, `gaiadr3.tmass_psc_xsc_best_neighbour` and `gaiadr3.tmass_psc_xsc_join`. This methodology is based heavily on the example provided in Sec 2.1.1 of [this link](#). The cone search condition is structured similarly to the example in Sec 1.1 of [this link](#). We require from our query `phot_g_mean_mag`, `ph_qual`, `parallax`, `bp_rp`, `j_m`, `h_m`, and `ks_m`. We constrain the results by `phot_g_mean_mag < 14` and RA/Dec within 1.0 degrees of the M67 coordinates.

```
SELECT
g3.source_id,  -- Gaia DR3 ID
tm.designation,  -- 2MASS ID
g3.ra,  -- Gaia RA
g3.dec,  -- Gaia Dec
g3.phot_g_mean_mag,  -- Gaia G mag
g3.parallax,  -- Gaia parallax
g3.bp_rp,  -- Gaia BP-RP
tm.ph_qual,  -- 2MASS photometry flag
tm.j_m,  -- 2MASS J mag
tm.h_m,  -- 2MASS H mag
tm.ks_m  -- 2MASS Ks mag

FROM gaiadr3.gaia_source AS g3

JOIN gaiadr3.tmass_psc_xsc_best_neighbour AS xm_t
  USING (source_id)

JOIN gaiadr3.tmass_psc_xsc_join AS join_t
  ON xm_t.original_ext_source_id = join_t.original_psc_source_id

JOIN gaiadr1.tmass_original_valid AS tm
  ON join_t.original_psc_source_id = tm.designation

WHERE
  DISTANCE(132.825, 11.8, g3.ra, g3.dec) < 1.0  -- cone search within 1 deg
  AND g3.phot_g_mean_mag < 14  -- HERMES limit (given in Q)
```

Python

I write the results to a file, `data/q1_xmatch_res.csv`, to prevent memory crashes losing the data locally and determine the data length.

```
# write table to .csv file without row numbers
xmatch_res.to_pandas().to_csv("data/q2_xmatch_res.csv", index=False)

data = pd.read_csv("data/q2_xmatch_res.csv") # read-in xmatch data
print("No. Stars: "+str(len(data))) # print len of data

# No. Stars: 1018
```

This ADQL query returns 1018 stars.

We remove stars with `ph_qual != "AAA"` and `parallax < 0` from our `xmatched` data (`pd.DataFrame()` named `data`).

```
# define array mask based on quality reqs
qual_mask = (
    (data["ph_qual"] == "AAA") & # ph_qual must equal "AAA" AND
    (data["parallax"] >= 0) # parallax greater than or equal to 0
)
# apply mask
clean_data = data[qual_mask]
```

Python

```
clean_data.to_csv("data/q2_clean.csv", index=False)
clean_data = pd.read_csv("data/q2_clean.csv") # read-in xmatch data
print("No. Clean Stars: "+str(len(clean_data))) # print len of data

# No. Clean Stars: 988
```

This quality cut remains 988 stars.

- (c) Create and save a two-panel figure showing: Gaia $BP - RP$ versus absolute G magnitude, using `bp_rp`; 2MASS $J - Ks$ versus apparent H magnitude, using `j_m`, `h_m`, and `ks_m`.

For clarity, I begin by defining my subplot x - and y -axis variables.

Python

```
x_g3 = clean_data["bp_rp"] # gaia subplot x-axis (BP-RP)
y_g3 = clean_data["phot_g_mean_mag"] # gaia y-axis (G)
x_tm = clean_data["j_m"] - clean_data["ks_m"] # 2mass x (J-Ks)
y_tm = clean_data["h_m"] # 2mass y (H)
```

I then create the 2-panel figure using `matplotlib.pyplot`.

Python

```
# define figure with two horizontally-stacked subplots
fig, axs = plt.subplots(1, 2, figsize=(12,6), sharex=False, sharey=False)

# gaia (left) panel
ax = axs[0]
ax.scatter(x_g3, y_g3, c="DarkRed", marker=".", s=10)
ax.set_title(r"Gaia $BP-RP$ vs $G$", fontsize=16) # panel title
ax.set_ylabel(r"Gaia $G$ [mag]", fontsize=12) # y axis label
ax.set_xlabel(r"Gaia $BP-RP$ [mag]", fontsize=12) # x label
ax.invert_yaxis() # reverse y-axis st brighter stars at top

# 2mass (right) panel
ax = axs[1]
ax.scatter(x_tm, y_tm, c="DarkRed", marker=".", s=10)
ax.set_title(r"2MASS $J-Ks$ vs $H$", fontsize=16)
ax.set_ylabel(r"2MASS $H$ [mag]", fontsize=12)
ax.set_xlabel(r"2MASS $J-Ks$ [mag]", fontsize=12)
ax.invert_yaxis()

plt.tight_layout() # minimise whitespace around/between panels
plt.savefig("figures/q2c_g3_v_tm.png", dpi=200) # save to figures/
plt.show() # display figure
```

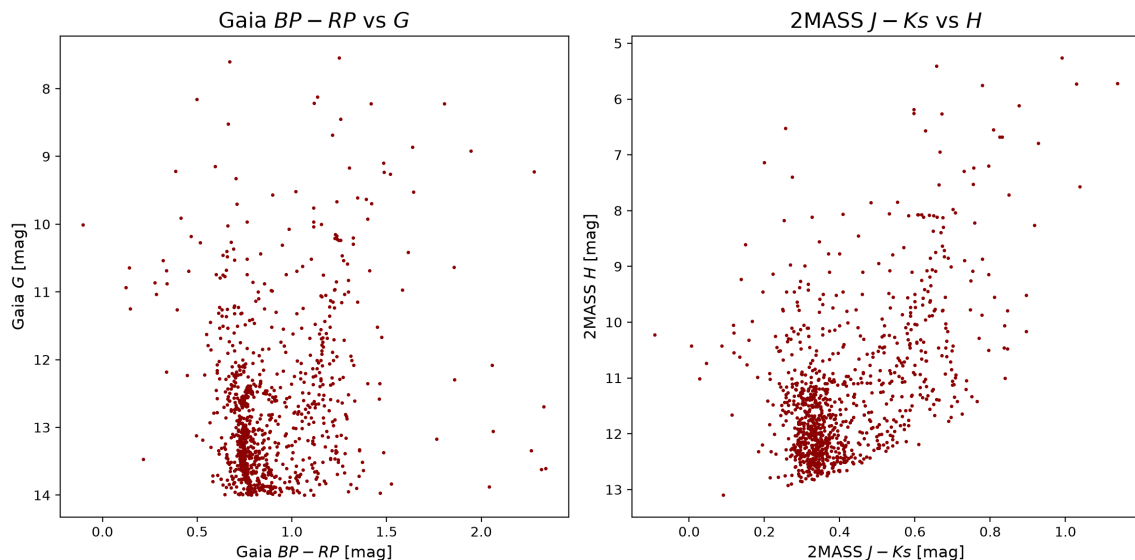


Figure 1: Resultant figure from python script.

- (d) Based only on fibre usage, would you recommend this field for a 2dF/HERMES observing proposal? Support your answer quantitatively.

By part (b), the M67 field contains 988 stars over a 1 deg FoV. **This link** tells us that 2dF/HERMES has a 2 deg FoV and ~ 350 science fibres. Simply, an observing proposal is suitable for $\lceil 988 / 350 \rceil = 3$ observations.

3 RADIAL METALLICITY RELATIONS IN SIMULATIONS

[6] The radial metallicity relation describes the change of metallicity - here, the gas-phase metallicity $A(\text{O}) = \log_{10}(N_{\text{O}}/N_{\text{H}}) + 12$ - which galactocentric cylindrical radius, R_{Gal} . Radial metallicity gradients provide insight into processes such as inside-out formation, gas accretion, outflows, and radial migration. A lot of work has been done through observational studies and a few simulations, but more work is needed to understand variations of the radial metallicity gradient.

Your colleague has completed a cosmological simulation of a MW-like spiral galaxy. The .fits file `data/nihao_uhd_simulation_g8.26_e11_xyz_positions_and_oxygen_ao.fits` contains the gas-phase metallicity $A(\text{O})$ and (x, y, z) positions of gas particles within $R_{\text{Gal}} = \sqrt{x^2 + y^2} < 25$ kpc. Perform the following tasks and save your figures in `figures/`.

- (a) Plot a two-panel figure: logarithmic density plot of R_{Gal} vs $A(\text{O})$, with a linear fit and a legend; residuals of the fit, R_{Gal} vs $\Delta A(\text{O})$.

I pull the data to my local repo `data/` folder using my terminal.

bash

```
wget -O data/q3_simulation_data.fits "https://raw.githubusercontent.com/svenbuder/astro_comp_assignment02_2026/main/data/nihao_uhd_simulation_g8.26_e11_xyz_positions_and_oxygen_ao.fits"
```

I import the necessary packages and read-in the data.

Python

```
# packages handling arrays
import numpy as np
# package for data visualisation
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm # for log cmaps
from matplotlib.colors import LinearSegmentedColormap
cmap_colours = ["mediumvioletred", "darkred"]
cmap = LinearSegmentedColormap.from_list(
    "cmap",
    ["white", cmap_colours[0], cmap_colours[1]]
) # set standard cmap
# astropy for handling data in .fits format
from astropy.io import fits
# packages for model fitting
import scipy as sp
import emcee as mc
# functions to format math in outputs
from IPython.display import display, Math

# read-in to file (containing header/metadata)
data_file = fits.open("data/q3_simulation_data.fits")
# extract data
data = np.array(data_file[1].data)
```

For clarity, I define the variables for both panels first.

Python

```
x = np.sqrt((data["x"])**2 + (data["y"])**2) # R_Gal (same for both panels)
y_main = data["A_0"] # A(O)
fit_res = np.polyfit(x, y_main, 1) # linear fit (deg=1), returns [m, b]
y_fit = (fit_res[0])*x + fit_res[1] # y = mx + b
y_resid = y_main - y_fit

# binning values
binsize = 100
counts_main, x_edges, y_edges_main = np.histogram2d(x, y_main, bins=binsize) # main histogram
counts_resid, _, y_edges_resid = np.histogram2d(x, y_resid, bins=binsize) # residuals histogram
```

I then plot and save a 2-subplot figure.

Python

```
# define figure with two vertically-stacked subplots
fig, axs = plt.subplots(2, 1, figsize=(10,8), sharex=True, sharey=False, gridspec_kw={
    "height_ratios": [2, 1], "hspace": 0})

# main (left) panel
ax = axs[0]
image = ax.pcolormesh(x_edges, y_edges_main, counts_main.T, cmap=cmap, norm=LogNorm(), shading=
    "auto") # plot 2D hist
ax.plot(x, y_fit, lw=1.5, marker="", c="black", label=r"${fit_res[0]:.3f}\cdot R_{\mathrm{{Gal}}}$"
    + ${fit_res[1]:.3f}$", ls="--", alpha=0.7)
```

```

ax.set_title(r"$A(\mathrm{O})$ vs $R_{\mathrm{Gal}}$", fontsize=16) # panel title
ax.set_ylabel(r"$A(\mathrm{O}) = \log(\mathrm{O}/\mathrm{H}) + 12$ [dex]", fontsize=12) # y
    axis label
ax.legend(loc="lower left", fontsize=13)

# residuals (right) panel
ax = axs[1]
ax.pcolormesh(x_edges, y_edges_resid, counts_resid.T, cmap=cmap, norm=LogNorm(), shading="auto"
)
ax.axhline(0, c="black", lw=1.5, ls="-.", alpha=0.7)
ax.set_ylabel(r"$\Delta A(\mathrm{O})$ [dex]", fontsize=12)
ax.set_xlabel(r"$R_{\mathrm{Gal}}$ [kpc]", fontsize=12) # x axis label
cbar = fig.colorbar(image, ax=axs, pad=0.01)
cbar.set_label("No. Particles", fontsize=12)

plt.savefig("figures/q3a_gasphase.png", dpi=200) # save to figures/
plt.show() # display figure

```

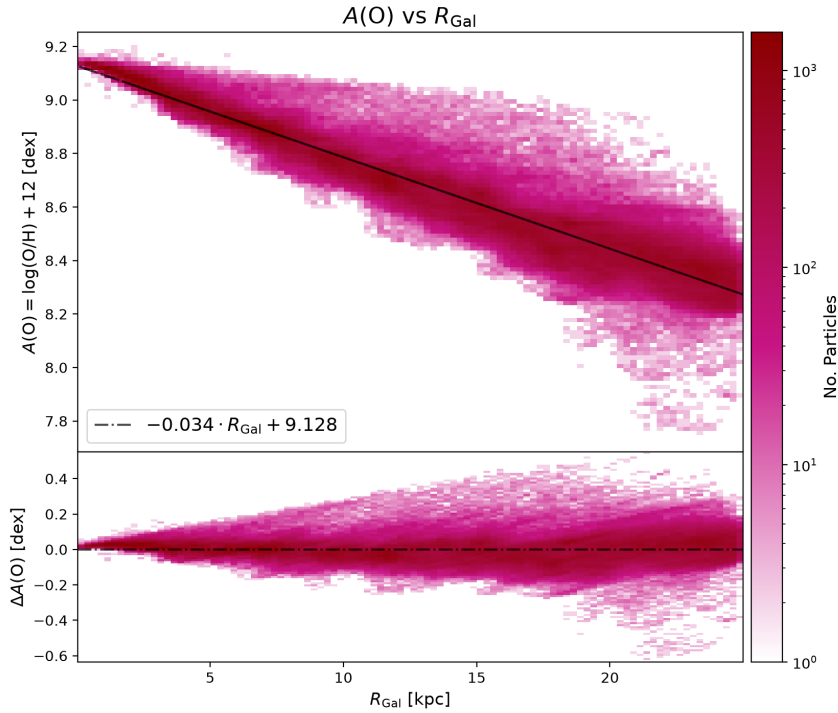


Figure 2: Resultant figure from python script.

- (b) Use a Python fitting tool to fit a linear function to the data, reporting the intercept and slope with uncertainties. Include any hyperparameters used.

I use a linear model including parameters, gradient/slope (m) and intercept (b), and hyperparameter, intrinsic scatter (σ_{int}). I select `emcee` as a suitable python tool to allow for convergence on a mean with gaussian uncertainties for each model parameter. I first construct the model, likelihood, prior, and posterior for the MCMC analysis. The priors are guided by the estimated linear relation in part (a), and the log likelihood formula for a normal distribution is taken from [this link](#).

Python

```

# linear relation
def model(theta, x):
    """
    Defines linear relationship between "A_0" and "R_Gal".
    Inputs:
        theta = model parameters (gradient/slope "m", intercept "b", log intrinsic scatter "
ln_sigma")
        x = input observable ("R_Gal")
    Outputs:
        y = output observable ("A_0")
    """

```

```

m, b, _ = theta
R_Gal = x
A_0 = m * R_Gal + b
return A_0

# parameters priors (advised by quick analysis in PART A)
def log_prior(theta):
    """
    Defines bounds for the MCMC walkers of each model parameter.
    Inputs:
        theta = model parameters (gradient/slope "m", intercept "b", log intrinsic scatter "
ln_sigma")
    Outputs:
        log probability (ln(1) = 0 if theta in bounds, ln(0) = -inf if not)
    """
    m, b, ln_sigma = theta
    if (-0.2 < m < 0.1) and (6 < b < 12) and (-5 < ln_sigma < 2):
        return 0.0 # ln(1)
    else:
        return -np.inf # ln(0)

# likelihood function
def log_likelihood(theta, x, y):
    """
    Calculate likelihood of observing each "R_Gal" and "A_0" given the model parameters.
    Inputs:
        theta = model parameters (gradient/slope "m", intercept "b", log intrinsic scatter "
ln_sigma")
        x = input observable ("R_Gal")
        y = output observable ("A_0")
    Outputs:
        log probability
    """
    _, _, ln_sigma = theta
    sigma = np.exp(ln_sigma) # de-log ln_sigma
    var = sigma**2 # variance

    y_model = model(theta, x) # model-predicted "A_0" values at each "R_Gal"
    resids = y - y_model # distance between model-predicted "A_0" and observed "A_0"

    ln_L = -0.5 * np.sum(np.log(2*np.pi*var) + resids**2/var)
    return ln_L

# posterior probability
def log_probability(theta, x, y):
    """
    Calculate the resultant probability of a model parameter set (including priors and
likelihood).
    Inputs:
        theta = model parameters (gradient/slope "m", intercept "b", log intrinsic scatter "
ln_sigma")
        x = input observable ("R_Gal")
        y = output observable ("A_0")
    Outputs:
        log probability (ln(0) = -inf if model rejected by priors, log likelihood if model
suitable since log prior = 0)
    """
    lp = log_prior(theta)
    if not np.isfinite(lp):
        return -np.inf # ln(0)
    else:
        ll = log_likelihood(theta, x, y)
        return ll # log likelihood

```

I use `scipy.stats.linregress()` to set realistic initial positions for the MCMC walkers.

Python

```

# scipy linear regression
fit_res = sp.stats.linregress(x, y_main) # input R_Gal and A(0)

m_init = fit_res.slope # initial gradient/slope
b_init = fit_res.intercept # initial intercept

resids = y_main - (m_init * x + b_init)

```

```

sigma_init = np.std(resids) # initial intrinsic scatter

# print initial values
print("Initial values:")
display(Math(f"$m = \{m\_init:.4f\}$"))
display(Math(f"$b = \{b\_init:.4f\}$"))
display(Math(rf"$\sigma_{\mathrm{{int}}} = \{\sigma\_init:.4f\}$"))

# Initial values:
# $\displaystyle m = -0.0342$
# $\displaystyle b = 9.1278$
# $\displaystyle \sigma_{\mathrm{{int}}} = 0.0726$

```

I then initialise the MCMC analysis through the `emcee` package.

Python

```

ndim = 3
nwalkers = 32 # > 2*ndim to sample sufficiently
nsteps = 5000

initial = np.array([m_init, b_init, np.log(sigma_init)]) # array of the central positions
perturbs = 1e-3 * np.random.randn(nwalkers, ndim) # define normal distribution of perturbations
positions = initial + perturbs # shift these perturbations to around central positions

# run MCMC sampling
sampler = mc.EnsembleSampler(nwalkers, ndim, log_probability, args=(x, y_main))
sampler.run_mcmc(positions, nsteps, progress=True)
samples = sampler.get_chain(
    discard=1000, # discard first positions of each walker
    flat=True # flatten nwalkers and nsteps into 1D array
)

```

From the flattened samples output, I extract the samples array for each of m , b , and σ_{int} .

Python

```

# read each samples array from full samples results
m_samples = samples[:,0]
b_samples = samples[:,1]
sigma_samples = np.exp(samples[:,2])

# extract percentiles
m_lo, m_med, m_hi = np.percentile(m_samples, [16, 50, 84])
b_lo, b_med, b_hi = np.percentile(b_samples, [16, 50, 84])
sigma_lo, sigma_med, sigma_hi = np.percentile(sigma_samples, [16, 50, 84])

# print results
display(Math(rf"$m = \{1e2*m\_med:.4f\}_{-\{1e2*(m\_med-m\_lo):.4f\}}^{+\{1e2*(m\_hi-m\_med):.4f\}} \times 10^{-2} \backslash, \mathrm{{dex}} \backslash, \mathrm{{kpc}}^{-1}$"))
display(Math(rf"$b = \{b\_med:.5f\}_{-\{(b\_med-b\_lo):.5f\}}^{+\{(b\_hi-b\_med):.5f\}} \backslash, \mathrm{{dex}}$"))
display(Math(rf"$\sigma_{\mathrm{{int}}} = \{1e2*\sigma\_med:.4f\}_{-\{1e2*(\sigma\_med-\sigma\_lo):.4f\}}^{+\{1e2*(\sigma\_hi-\sigma\_med):.4f\}} \times 10^{-2} \backslash, \mathrm{{dex}}$"))

# $\displaystyle m = -3.4196_{-0.0014}^{+0.0015} \times 10^{-2} \backslash, \mathrm{{dex}} \backslash, \mathrm{{kpc}}^{-1}$
# $\displaystyle b = 9.12783_{-0.00023}^{+0.00023} \backslash, \mathrm{{dex}}$
# $\displaystyle \sigma_{\mathrm{{int}}} = 7.2650_{-0.0072}^{+0.0070} \times 10^{-2} \backslash, \mathrm{{dex}}$

```

The computed values are therefore

$$m = (-3.1496_{-0.0014}^{+0.0015}) \times 10^{-2} \text{ dex kpc}^{-1} \quad b = (9.12783 \pm 0.00023) \text{ dex} \quad \sigma_{\text{int}} = (7.2646_{-0.0072}^{+0.0070}) \times 10^{-2} \text{ dex},$$

which is much inline with my estimates from both `numpy` and `scipy`, and where $\pm$ is used here for symmetric 1σ uncertainties.

- (c) Discuss where the linear model fits well and where it does not. Use statistical metrics to quantify its performance overall and in regions with larger residuals.

I compute the root mean square error to quantify the size of the residuals from the models:

$$\text{RMSE} = \sqrt{\left\langle \sum_i (y_{\text{sim},i} - y_{\text{model},i})^2 \right\rangle}.$$

This is selected since there are no provided uncertainties in the data to compute a χ^2 value.

Python

```
# compute MCMC residuals
mcmc_y = m_med * x + b_med
mcmc_resids = y_main - mcmc_y

# compute RMSE
rmse = np.sqrt(np.mean(mcmc_resids**2))
print(f"RMSE = {rmse:.4f} dex")

# RMSE = 0.0726 dex
```

I find an overall RMSE to be 0.073, in much agreement with my MCMC-computed σ_{int} . I then determine this RMSE over radial bins of width 500 pc to quantify the radial dependence of the fit quality.

Python

```
# define radial bins of width 0.5kpc
bins = np.arange(0, 25.5, 0.5)

# determine rmse in each bin
centre_array = []
rmse_array = []
for r_min, r_max in zip(bins[:-1], bins[1:]):
    mask = (x >= r_min) & (x < r_max) # only include partial in given radial bin
    rmse_bin = np.sqrt(np.mean(mcmc_resids[mask]**2))
    centre_array.append(np.mean([r_min, r_max])) # bin centres
    rmse_array.append(rmse_bin)
centre_array = np.array(centre_array)
rmse_array = np.array(rmse_array)
```

For clarity, I then plot my results using the following code.

Python

```
fig, axs = plt.subplots(2, 1, figsize=(10,8), sharex=True, sharey=False, gridspec_kw={"
    height_ratios": [2, 2], "hspace": 0})

ax = axs[0] # main (left) panel
image = ax.pcolormesh(x_edges, y_edges_main, counts_main.T, cmap=cmap, norm=LogNorm(), shading=
    "auto", alpha=0.7) # plot 2D hist
ax.plot(x, y_fit, lw=1.5, marker="", c="black", label=r"${fit_res[0]:.3f}\cdot R_{\mathrm{{Gal}}}$
    } + ${fit_res[1]:.3f}$", ls="-.", alpha=0.7)
ax.fill_between(centre_array, (m_med*centre_array+b_med)-rmse_array, (m_med*centre_array+b_med)
    +rmse_array, color="black", alpha=0.3, edgecolor=None, label="RMSE")
ax.set_title(r"$A(\mathrm{{0}})$ vs $R_{\mathrm{{Gal}}}$", fontsize=16) # panel title
ax.set_ylabel(r"$A(\mathrm{{0}}) = \log(\mathrm{{0}}/\mathrm{{H}}) + 12$ [dex]", fontsize=12) # y
    axis label
ax.legend(loc="lower left", fontsize=13)

ax = axs[1] # residuals (right) panel
ax.pcolormesh(x_edges, y_edges_resid, counts_resid.T, cmap=cmap, norm=LogNorm(), shading="auto"
    , alpha=0.7)
ax.axhline(0, c="black", lw=1.5, ls="-.", alpha=0.7)
ax.fill_between(centre_array, -rmse_array, rmse_array, color="black", alpha=0.3, edgecolor=None
    , label="RMSE")
ax.set_ylabel(r"$\Delta A(\mathrm{{0}})$ [dex]", fontsize=12)
ax.set_xlabel(r"$R_{\mathrm{{Gal}}}$ [kpc]", fontsize=12) # x axis label
ax.set_ylim(-0.25, 0.25)
cbar = fig.colorbar(image, ax=axs, pad=0.01)
cbar.set_label("No. Particles", fontsize=12)

plt.savefig("figures/q3c_residuals.png", dpi=200) # save to figures/
plt.show() # display figure
```

It becomes clear in this figure that the data deviates further from a linear fit with increased Galactocentric radius, where the RMSE increases from ~ 0.025 dex to ~ 0.09 dex.

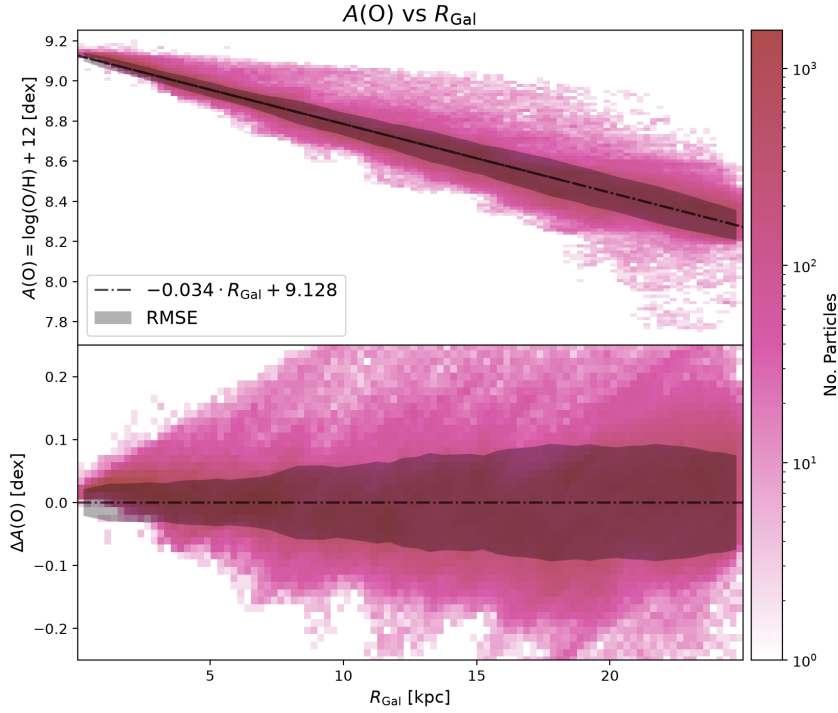


Figure 3: Resultant figure from python script.

- (d) Plot a three-panel figure for the x vs y plane using the same bins and sensible colormaps: 2D histograms of the median simulated $A(O)$; 2D histogram of the median fitted $A(O)$; 2D histogram of the median residuals $\Delta A(O)$.

I compute the median in each (x, y) bin using `scipy.stats.binned_statistic_2d()` with `statistic="median"`. I did this also for the linear fit (with $R_{\text{Gal}} = \sqrt{x^2 + y^2}$ and the residuals $y_{\text{residual}} = y_{\text{sim}} - y_{\text{fit}}$).

Python

```
x_kpc = data["x"] # x (same for all panels)
y_kpc = data["y"] # y (same for all panels)
ao = data["A_0"]
ao_fit = m_med * np.sqrt((data["x"])**2 + (data["y"])**2) + b_med # computed again for self-
    consistency within subpart (using PART B)
ao_resids = ao - ao_fit # residuals from fit

# extract bin-median A(O) for each, bin x-axis edges, bin y-axis edges, bin number
binsize = 100 # define number of bins
median_ao_data, x_edges, y_edges, bin_no = sp.stats.binned_statistic_2d(x_kpc, y_kpc, ao,
    statistic="median", bins=binsize) # panel 1
median_ao_fit, _, _, _ = sp.stats.binned_statistic_2d(x_kpc, y_kpc, ao_fit, statistic="median",
    bins=[x_edges, y_edges]) # panel 2
median_ao_resids, _, _, _ = sp.stats.binned_statistic_2d(x_kpc, y_kpc, ao_resids, statistic="
    median", bins=[x_edges, y_edges]) # panel 3
```

I plot two panels with the simulated data and fit, then the residuals are plotted with an extended divergent colourbar (same as `cmap` with blue gradient into negatives).

Python

```
# define figure with three horizontally-stacked subplots
fig, axs = plt.subplots(1, 3, figsize=(15,5), sharex=True, sharey=True, constrained_layout=True)

# set colorbar limits
vmin = np.nanmin([median_ao_data, median_ao_fit]) # find minimum values excluding NaNs
vmax = np.nanmax([median_ao_data, median_ao_fit]) # max excl NaNs
residsmax = np.nanmax(np.abs(median_ao_resids)) # abs max residuals value

# define residual (diverging) cmap
div_cmap = LinearSegmentedColormap.from_list(
    "div_cmap",
```

```

["darkblue", "mediumblue", "white", cmap_colours[0], cmap_colours[1]]
)

ax = axs[0] # simulation (left) panel
image = ax.pcolormesh(x_edges, y_edges, median_ao_data.T, vmin=vmin, vmax=vmax, cmap=cmap,
    shading="auto")
ax.set_title(r"Simulated  $A(\mathrm{O})$ ", fontsize=16) # panel title
ax.set_ylabel(r"$y$ [kpc]", fontsize=12) # y axis label
ax.set_xlabel(r"$x$ [kpc]", fontsize=12) # x label

ax = axs[1] # linear fit (centre) panel
image = ax.pcolormesh(x_edges, y_edges, median_ao_fit.T, vmin=vmin, vmax=vmax, cmap=cmap,
    shading="auto")
ax.set_title(r"Linear Fit  $A(\mathrm{O})$ ", fontsize=16) # panel title
ax.set_xlabel(r"$x$ [kpc]", fontsize=12) # x label
cbar1 = fig.colorbar(image, ax=axs[2], pad=0.01)
cbar1.set_label(r"Median  $A(\mathrm{O})$  [dex]", fontsize=12)

ax = axs[2] # residuals (right) panel
image = ax.pcolormesh(x_edges, y_edges, median_ao_resids.T, vmin=-residsmax, vmax=residsmax,
    cmap=div_cmap, shading="auto")
ax.set_title(r"Residual  $A(\mathrm{O})$ ", fontsize=16) # panel title
ax.set_xlabel(r"$x$ [kpc]", fontsize=12) # x label
cbar1 = fig.colorbar(image, ax=axs[2], pad=0.01)
cbar1.set_label(r"Median Residuals  $\Delta A(\mathrm{O})$  [dex]", fontsize=12)

# plt.tight_layout() # minimise whitespace around/between panels
plt.savefig("figures/q3d_galactic.png", dpi=200) # save to figures/
plt.show() # display figure

```

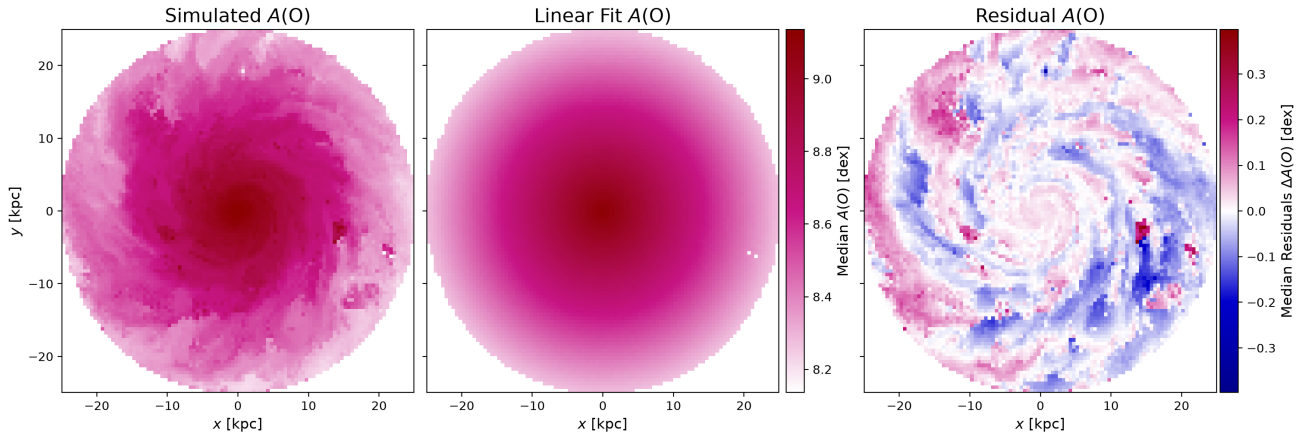


Figure 4: Resultant figure from python script.

- (e) Describe your choice of 2D bins. Discuss what details would be missed with fewer bins and what problems could arise with more bins.

I elected to take the median in 100 bins in both axes. With fewer bins, I would lose the local galactic features (eg, spiral arms, starforming/high- $A(\mathrm{O})$ regions). With more bins, the global trend of a general decrease in $A(\mathrm{O})$ is lost to only some diffuse spiral structure.

- (f) Analyse the residuals in more detail and propose a reasonable physical explanation any patterns you observe.

I observe the following patterns.

- A spiral structure emerges beyond $R_{\mathrm{Gal}} \sim 5 \text{ kpc}$ with distinct metal-rich and metal-poor arms. Each simulated metal-rich arm might coincide with a physical spiral arm, where SFRs are typically higher and the greater metal production remains mostly bound to the arm. The spiral structure becomes more pronounced at higher R_{Gal} , contributing to the higher measured RMSE.
- Some clumping of metal-rich regions is visible at $\sim (15, 0)$, $(20, -5)$, $(-10, -5)$. The clumped, high-O abundance regions have $r \sim 250 \text{ pc}$, similar to expanding ejecta from the CCSNe in which O tends to synthesise.

4 STELLAR SPECTRUM REDUCTION

Veloce is a high-resolution echelle spectrograph. The `flat.fits` and `moon.fits` in `data/` contain the same 4112×385 pixel detector region around the CaII triplet ($\sim 851 \rightarrow 871$ nm). The dispersion direction is vertical and the cross-dispersion direction, horizontal. The central science fibres are bounded by two low-illumination gaps; the curves following these gaps are the *tramlines*. Signal from neighbouring spectra orders have been set to zero. You may assume that the distance between the two tramlines is constant.

4.1 DETERMINE THE TRAMLINES.

[2] Use the flat-field image to determine the tramline positions as a function of dispersion pixel and fit an appropriate smooth function. Briefly explain your approach and plot the flat with your fitted tramlines overlaid.

I pull the data to my local repo `data/` folder using my terminal.

```
bash
```

```
wget -O data/q4_moon.fits "https://raw.githubusercontent.com/svenbuder/astro_comp_assignment02_2026/main/data/moon.fits"
wget -O data/q4_flat.fits "https://raw.githubusercontent.com/svenbuder/astro_comp_assignment02_2026/main/data/flat.fits"
```

I import the necessary packages.

```
Python
```

```
# packages handling arrays
import numpy as np
import pandas as pd
# package for data visualisation
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
cmap_colours = ["mediumvioletred", "darkred"]
cmap = LinearSegmentedColormap.from_list(
    "cmap",
    ["white", cmap_colours[0], cmap_colours[1]]
) # set standard cmap
# astropy for handling data in .fits format
from astropy.io import fits
# packages for model fitting
import scipy as sp
```

I read-in the `flat.fits` data and plot this using `plt.imshow()`. I include an example spectrum over the cross-dispersion axis, at pixel 1500 of the dispersion axis. This plot is for my visualisation and debugging purposes, and can be viewed in 4.ipy nb.

```
Python
```

```
# read-in to file (containing header/metadata)
flat_file = fits.open("data/q4_flat.fits")
# extract data
flat = np.array(flat_file[0].data)

fig, axs = plt.subplots(2, 1, figsize=(8, 7), sharex=True, sharey=False, gridspec_kw={"height_ratios": [3, 1], "hspace": 0})

ax = axs[0] # top panel (full flat)
image = ax.imshow(flat*1e-4, cmap=cmap, origin="lower", aspect="auto")
ax.set_title("Flat Spectrum", fontsize=16)
ax.set_xlabel("Cross-Dispersion [pixels]", fontsize=12)
ax.set_ylabel("Dispersion [pixels]", fontsize=12)

ax = axs[1] # bottom panel (spectrum extraction at pixel=1500)
ax.plot(np.arange(len(flat[1500,:])), np.array(flat[1500,:])*1e-4, c="black", lw=0.8)
ax.text(20, 1.4, "Dispersion Pixel 1500", c="black")
ax.set_xlabel("Cross-Dispersion [pixels]", fontsize=12)
ax.set_ylabel(r"Electron Count [ $\times 10^4$ ]", fontsize=12)
cbar = fig.colorbar(image, ax=axs, pad=0.01)
cbar.set_label(r"Electron Count [ $\times 10^4$ ]", fontsize=12)

plt.show()
```

To determine the positions of the tramlines, I isolate the cross-dispersion spectrum at each dispersion pixel. I introduce a noise threshold at an electron count of 1000, and ignore all the dispersion pixels that remain below this values throughout their entire spectrum. I then identify the main signal (the streak across the plot) by isolating cross-dispersion pixels that are at least 50% of the max intensity of the spectrum; dispersion pixels with fewer than 45 pixels satisfying this condition (to avoid including noise) are ignored. Finally, I invert each spectrum along the

cross-dispersion axis and use `sp.signal.find_peaks()` to identify maxima within the streak and with a separation of at least 25 pixel (to ensure artefacts are not considered peaks). If there are more or fewer than 2 peaks, this dispersion pixel is ignored. Each of these discarded pixels is to safeguard this automated process against noise/false signals.

I then define a `pd.DataFrame()` with the array indices of the left and right tramlines. I overplot the points of these detected peaks to visually inspect that the identified peaks align with the tramlines.

Python

```
first_peak = []
second_peak = []

for i in np.arange(flat.shape[0]): # iterate over every dispersion pixel

    # extract cross-dispersion spectrum for each dispersion pixel
    spec = np.array(flat[i,:])

    # define threshold cut to isolate signal for peaks detection
    spec_max = np.nanmax(spec) # spectrum max intensity
    noise_thres = 1000 # electron count minimum to be not noise
    if spec_max < noise_thres:
        first_peak.append(np.nan)
        second_peak.append(np.nan)
        continue

    detect_thres = 0.5 # pixel must have 50% of spectrum max intensity
    valid_pixels = (np.where(spec > spec_max*detect_thres))[0]
    if len(valid_pixels) < 45: # detected signal must be at least 100 pixels wide
        first_peak.append(np.nan)
        second_peak.append(np.nan)
        continue

    signal = spec[valid_pixels[1]:valid_pixels[-1]]
    peaks = np.array((sp.signal.find_peaks(-signal, distance=25))[0]) + valid_pixels[1]
    if len(peaks) != 2: # must find exactly 2 peaks
        first_peak.append(np.nan)
        second_peak.append(np.nan)
        continue

    first_peak.append(peaks[0]) # left tramline
    second_peak.append(peaks[1]) # right tramline

tramlines = pd.DataFrame({
    "left": np.array(first_peak), # x-pos left tramline
    "right": np.array(second_peak), # y-pos right tramline
    "disp": np.arange(flat.shape[0]) # y-pos
})

plt.figure(figsize=(8,6))

image = plt.imshow(flat*1e-4, cmap=cmap, origin="lower", aspect="auto", alpha=0.7) # plot full flat
plt.scatter(np.array(tramlines["left"]), np.array(tramlines["disp"]), c="black", s=0.5) # left
    tramline from identified peaks
plt.scatter(np.array(tramlines["right"]), np.array(tramlines["disp"]), c="black", s=0.5) # right
    tramline from identified peaks
plt.title("Flat Spectrum with Peaks", fontsize=16)
plt.xlabel("Cross-Dispersion [pixels]", fontsize=12)
plt.ylabel("Dispersion [pixels]", fontsize=12)
cbar = plt.colorbar(image, pad=0.01)
cbar.set_label(r"Electron Count [ $\times 10^4$ ]", fontsize=12)

plt.show()
```

Finally, I use `sp.interpolate.UnivariateSpline()` to fit a function to the identified peaks; this is a function of dispersion pixel since the interpolated function must be injective. I use a high `s`-parameter to avoid overfitting the function to the peak points. I create an array along the dispersion axis (`disp_smooth`) which the function is then computed over to create smooth tramlines (`left_smooth` and `right_smooth`). Finally, I overplot these tramline functions.

Python

```
# remove any dispersion pixel spectrum with NaN tramlines before fitting function
tramlines = tramlines.dropna(axis=0, how="any")

# define numpy arrays
disp_pixel = np.array(tramlines["disp"])
left_tram = np.array(tramlines["left"])
right_tram = np.array(tramlines["right"])
```

```

# fit to function
left_spline = sp.interpolate.UnivariateSpline(displ_pixel, left_tram, s=1200) # very high s to
    minimise influence of noise
right_spline = sp.interpolate.UnivariateSpline(displ_pixel, right_tram, s=1200)

disp_smooth = np.arange(flat.shape[0])

left_smooth = np.array(left_spline(disp_smooth))
right_smooth = np.array(right_spline(disp_smooth))

plt.figure(figsize=(8,6))

image = plt.imshow(flat*1e-4, cmap=cmap, origin="lower", aspect="auto", alpha=0.7) # full flat
plt.plot(left_smooth, disp_smooth, c="black", lw=2) # overplot tramlines
plt.plot(right_smooth, disp_smooth, c="black", lw=2)
plt.title("Flat Spectrum with Smooth Tramlines", fontsize=16)
plt.xlabel("Cross-Dispersion [pixels]", fontsize=12)
plt.ylabel("Dispersion [pixels]", fontsize=12)
cbar = plt.colorbar(image, pad=0.01)
cbar.set_label(r"Electron Count [ $\times 10^4$ ]", fontsize=12)

plt.savefig("figures/q4a_tramlines.png", dpi=200) # save to figures/
plt.show()

```

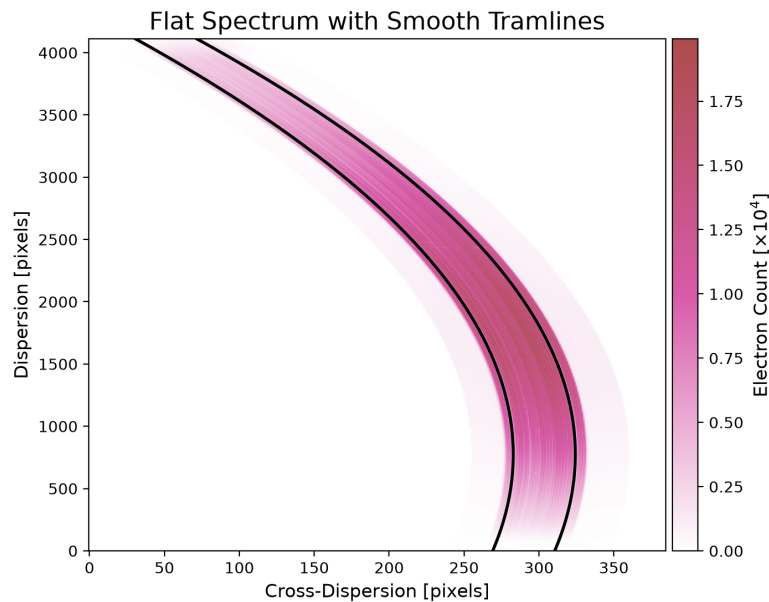


Figure 5: Resultant figure from python script.

4.2 EXTRACT THE SPECTRUM.

[4] Sum the counts between your tramlines to extract the flat-field and Moon spectra. Divide the Moon by the flat and plot the result as a function of dispersion pixel. Identify and indicate the pixels corresponding to the CaII lines at 8498, 8542, and 8662 Å.

I read-in the moon.fits spectrum and sum both spectra between the tramlines fitted in part (a). Since the cross-dispersion pixels and the tramlines are defined on opposite axes, I project both to a 2D array and set each position array[i,j] to True if between the tramlines or False if not. I then apply this mask, broadcasting NaN values to each position with False. Over the cross-dispersion axis, I np.nansum() to ignore NaN values and sum only values with positions set to True.

Python

```

# read-in to file (containing header/metadata)
moon_file = fits.open("data/q4_moon.fits")
# extract data
moon = np.array(moon_file[0].data)

xdisp_array = np.arange(flat.shape[1]) # x-dispersion axis

# create mask to only include values between the tramlines

```

```

mask = (
    (xdisp_array[np.newaxis,:] > left_smooth[:,np.newaxis]) & # project x-disp array and left
    # tramline to 2D arrays
    (xdisp_array[np.newaxis,:] < right_smooth[:,np.newaxis])
)

# apply mask, does not require constant distance between tramlines
flat_array = np.where(mask, flat, np.nan) # return flat pixel value if mask=True, return NaN if mask
# =False
moon_array = np.where(mask, moon, np.nan)

# sum over x-disp axis ignoring NaNs (broadcast to all values outside tramlines)
flat_sum = np.nansum(flat_array, axis=1)
moon_sum = np.nansum(moon_array, axis=1)

```

I identify the pixel ratios between the short- λ line interval and the long- λ line interval. This allows me to determine which lines are most likely to be the Ca II triplet and what the wavelength-direction of the dispersion axis is.

Python

```

lo_ca = 8662 # long-lambda line, values taken from Q
mid_ca = 8542 # middle line
hi_ca = 8498 # short line
ratio = np.abs((hi_ca - mid_ca) / (mid_ca - lo_ca))
print(f"Short-lambda interval is {ratio:.1f} times long-lambda interval.")

# Short-lambda interval is 0.4 times long-lambda interval.

```

From this, the dispersion axis increases with shorter wavelength. I divide the summed moon spectrum by the summed flat spectrum to compute the normalised spectrum. Thus, the normalised spectrum is below, with Ca II triplet labelled.

Python

```

norm_spec = np.array(moon_sum / flat_sum) # normalised spectrum
disp_array = np.arange(flat.shape[0]) # dispersion axis

plt.figure(figsize=(8,5))

plt.plot(disp_array, norm_spec, c="darkred", lw=1) # normalised spectrum
plt.text(3520, 0.225, r"$8498\AA$", size=13, weight="bold") # label short-lambda
plt.text(2860, 0.14, r"$8542\AA$", size=13, weight="bold") # label middle
plt.text(520, 0.15, r"$8662\AA$", size=13, weight="bold") # label long
plt.title("Normalised Veloce Moon Spectrum")
plt.xlabel("Dispersion [pixels]")
plt.ylabel("Normalised Electron Count (Moon / Flat)")

plt.savefig("figures/q4b_spectrum.png", dpi=200) # save to figures/
plt.show()

```

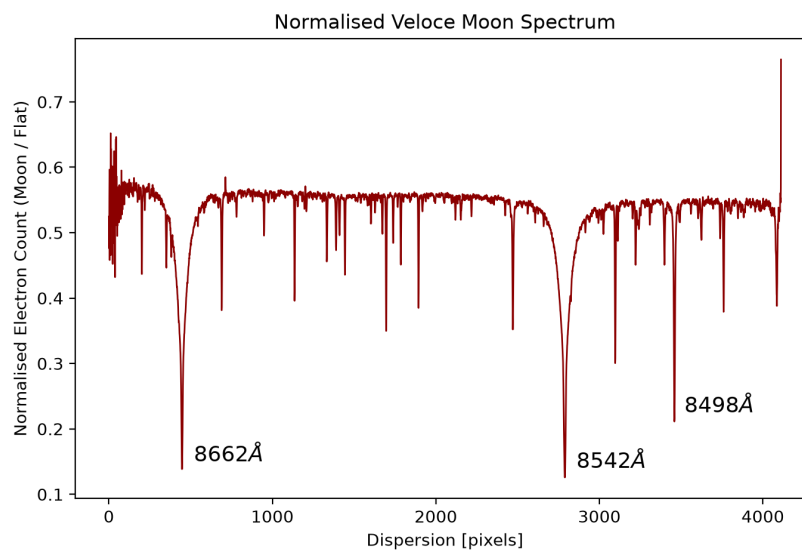


Figure 6: Resultant figure from python script.